

0. SETUP (1ª CÉLULA)

```
import pandas as pd
postos = pd.read_csv("postos_sp_2025.csv", sep=";",
decimal=",")
distrib = pd.read_csv("distribuicao_municipios_sp_2025.csv",
sep=";", decimal=",")

# Tira espaço fantasma do começo do CNPJ (defeito do CSV da
ANP)
postos["CNPJ"] = postos["CNPJ"].str.strip()

# Alternativa: strip em todas colunas de texto de uma vez
# for col in postos.select_dtypes(include="object").columns:
#     postos[col] = postos[col].str.strip()

postos["Mes"] = (postos["Data da Coleta"].str[6:10] + "-"
+ postos["Data da Coleta"].str[3:5])
print(postos.shape, distrib.shape); postos.head()
```

CSV BR: sempre sep=";" + decimal=",". Sem isso, parse quebra ou número vira string.

1. INSPEÇÃO

df.head(n) tail() sample()	linhas
df.shape (sem ())	(lin, col)
df.info() dtypes columns	tipos/nomes
df.describe()	num: count/mean/std/min/Q/max
describe(include="all")	+ texto
len(df)	nº linhas

Pra cada DataFrame novo, sempre roda shape+info()+head() ANTES de qualquer análise. Confere tamanho, tipo das colunas, e amostra visual. info() mostra se algum número virou string (problema do decimal).

2. SELEÇÃO DE COLUMNA

df["c"]	# Series (1D)
df[["c1","c2"]]	# DataFrame (várias, ordem da lista)
df[["c"]]	# DataFrame com 1 col (≠ Series)

1 colchete vira Series, 2 colchetes viram DataFrame. Pra reordenar colunas, usa 2.

DataFrame é a tabela inteira (2D, várias colunas). Series é uma coluna sozinha (1D). Series tem métodos de coluna (mean, str, value_counts). DataFrame não. 99% das operações tu quer Series (1 colchete).

3. FILTRO LÓGICO (LINHA)

```
cond = df["Produto"] == "GASOLINA" # Series booleana
df[cond] # filtra

df[df["Produto"] == "GASOLINA"]
df[df["Valor de Venda"] > 6]
df[df["Bandeira"] != "BRANCA"]
df[df["Bandeira"].isin(["VIBRA","RAIZEN"])]
len(df[df["Produto"]=="GASOLINA"]) # quantas
```

Filtro é sempre 2 etapas: cria Series booleana (True/False por linha) com a condição, aplica dentro do df[...]. Pandas vê booleana ali e fica só com linhas True. Ops: == != > < >= <= e .isin([...]) pra "está na lista".

4. AND / OR / NOT

```
# NUNCA and/or/not. Sempre & | ~
df[(df["Produto"]=="GASOLINA") & (df["Valor de Venda"]>6)]
df[(df["Bandeira"]=="VIBRA") | (df["Bandeira"]=="RAIZEN")]
df[~(df["Bandeira"]=="BRANCA")]

# Recomendado (cada cond como variável):
c1 = df["Produto"]=="GASOLINA"
c2 = df["Valor de Venda"]>6
df[c1 & c2]
```

Cada condição entre parênteses. Esquecer é erro #1.

Pandas tem operadores próprios pra booleanas: & (E), | (OU), ~ (NÃO). NUNCA usa and/or/not de Python puro. Cada condição entre parênteses, ou a precedência quebra silenciosamente.

5. LOC / ILOC

```
df.loc[cond, "Valor de Venda"]
df.loc[cond, ["Revenda","Valor de Venda"]]
df.loc[df["Produto"]=="GASOLINA","Valor de Venda"].mean()
df.iloc[0] # 1ª linha (posição)
df.iloc[0:5, 0:3] # FIM EXCLUSIVO
df.loc[5:10] # FIM INCLUSIVO
```

	loc	iloc
Usa	rótulo	posição
Fim	inclusivo	exclusivo

Pra filtrar linha E selecionar coluna ao mesmo tempo, OBRIGATÓRIO usar loc. Sem ele, dá warning. Vírgula separa: antes é seletor de linha, depois é seletor de coluna.

6. .STR (STRINGS NA COLUMNA)

```
df["c"].str.upper() / .lower() / .title() / .strip()
df["c"].str.len() # tamanho
df["c"].str.contains("POSTO") # booleana (filtro!)
df["c"].str.startswith("AUT0") / .endswith("LTDA")
df["c"].str.replace("LTDA","") # SUBSTRING
df["c"].str[0] .str[-3:] .str[6:10] # slice
df["c"].str.split() # vira lista
df["c"].str.split(expand=True) # vira DF
df["c"].str.split().str[-1] # último elemento
```

```
df["c"].str.upper().str.replace("LTDA","").str.strip() #
encadeia
```

Sem .str: 'Series' object has no 'upper'. Sempre .str antes.

.str é a ponte entre Series e métodos de string Python. Sem ela, df["c"].upper() falha porque Series não tem upper. Com .str, o método é aplicado em cada string da coluna, retorna Series nova.

7. REPLACE VS .STR.REPLACE

Forma	Compara
col.replace("SP","BA")	valor inteiro
col.str.replace("SP","BA")	substring no texto

```
df["Estado"] = df["Estado"].replace("SP","Sao Paulo")
df["Bandeira"] =
df["Bandeira"].replace({"BRANCA":"SEM","VIBRA":"BR"})
```

replace compara o valor INTEIRO (célula tem que ser exatamente "SP"). .str.replace troca SUBSTRING dentro do texto (parcial). Dicionário {velho: novo} troca vários valores de uma vez.

8. CRIAR / ATUALIZAR COLUMNA

```
df["nova"] = df["a"] + df["b"] # cria à direita
df["a"] = df["a"] * 1.1 # substitui
df["Pais"] = "Brasil" # constante
df["Cid_U"] = df["Município"].str.upper()
```

Pegadinha mortal: df = df["c"].str.lower() DESTRÓI o DF (vira Series). Esquerda tem que ser df["c"], NUNCA df.

Pra criar OU atualizar coluna, esquerda é df["c"]. Se a coluna já existe, substitui; se não, cria à direita da última. Mesma sintaxe pros dois casos.

9. ASTYPE (CONVERTER TIPO)

```
df["c"] = df["c"].astype(float / str / int)
```

```
# Se Valor veio string (decimal não pegou):
df["Valor de Venda"] = (df["Valor de Venda"]
.str.replace(",",".").astype(float))
```

String + número falha. Converte: "C" + df["id"].astype(str).

Converte o tipo da coluna inteira. Usa quando pandas leu errado (número virou string) ou quando precisa concatenar/operar tipos diferentes.

10. ESTATÍSTICA NUMÉRICA

```
df["c"].mean() .median() .std() .var()
df["c"].min() .max() .sum() .count() # count = não-
nulos
df["c"].quantile(0.25) .quantile(0.75)
df["c"].describe() # tudo de uma vez
df["c"].idxmax() # ÍNDICE do maior
```

Sem (): df["c"].mean retorna bound method, não o número.

Toda estatística aplicada em UMA coluna específica, retorna 1 número. df["c"].mean(), não df.mean(). idxmax retorna o ÍNDICE (posição/rótulo) onde tá o máximo, não o valor.

